

Relatório de Análise e Decisão Arquitetural - Aula 06

PharmaLink - Estilos Arquiteturais, Requisitos de Qualidade e Trade-offs

Campo	Preenchimento
Projeto	PharmaLink
Domínio	Farmácia digital com medicamentos, estoque, pedidos, pagamento, entrega, fornecedores e notificações
Integrantes	Michelly Lima, Ryan Rodrigues, Júlyya Andrade e Guilherme Felipe
Representante	Michelly Lima
Data	08/09/2026

Sumário

1. Projeto e domínio
 2. Arquitetura atual
 - 2.1 Diagrama da arquitetura atual
 - 2.2 Evidências no projeto
 3. Requisitos funcionais
 4. Requisitos não funcionais
 5. Relação RNF x parte afetada
 6. Problema arquitetural encontrado
 7. Matriz de decisão arquitetural
 - 7.1 Justificativa das notas
 8. Decisão, trade-off e proposta
 - 8.1 Diagrama da arquitetura proposta
 9. Conclusão do grupo
- Checklist antes da entrega

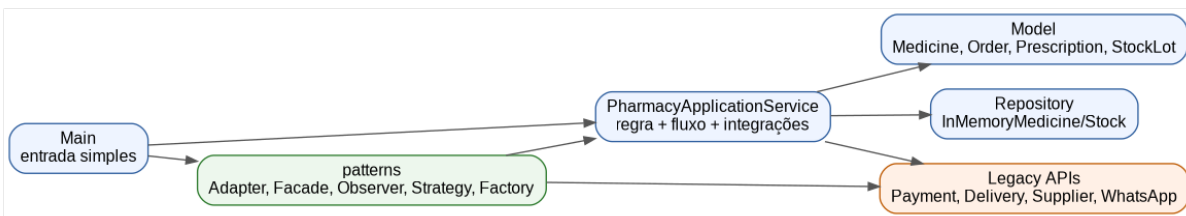
1. Projeto e domínio

O PharmaLink simula uma farmácia digital. O sistema recebe pedidos, consulta medicamentos e estoque, valida situações de receita, calcula desconto, processa pagamento, aciona entrega, conversa com fornecedores e envia notificações ao cliente. O projeto fornecido é um sistema legado em evolução: executa em Java 17, mas possui decisões didáticas incompletas e pontos arquiteturais a serem avaliados.

2. Arquitetura atual

A arquitetura real identificada no código é um monólito simples em camadas, com entrada por classe Main, serviço de aplicação central, modelos de domínio, repositórios em memória, integrações legadas e alguns padrões de projeto em pacotes separados. A descrição antiga do projeto declara camadas e preparação para microsserviços, mas o código ainda não possui fronteiras, contratos e implantação independente para sustentar essa afirmação.

2.1 Diagrama da arquitetura atual



2.2 Evidências no projeto

Elemento arquitetural	Evidência no projeto	Observação
Entrada / apresentação	src/main/java/br/edu/pharmalink/Main.java	Cria repositórios, serviço, dados de exemplo e executa o fluxo principal.
Serviço central	service/PharmacyApplicationService.java	Concentra cadastro, pedido, estoque, pagamento, entrega, WhatsApp, desconto e publicação de eventos.
Modelo de domínio	model/Medicine.java, Order.java, Prescription.java, StockLot.java	Entidades simples com atributos públicos e poucas regras encapsuladas.
Persistência em memória	repository/InMemoryMedicineRepository.java e InMemoryStockRepository.java	Armazenamento temporário usando Map e List.
Integrações legadas	legacy/PaymentLegacyGateway.java, DeliveryPartnerApi.java, SupplierLegacyApi.java, WhatsappLegacyApi.java	Classes simulam sistemas externos. O serviço central depende diretamente de algumas delas.
Padrões existentes	patterns/adapter, facade, observer, factory, strategy	Existem padrões, mas alguns ainda não isolam totalmente o domínio das integrações.

3. Requisitos funcionais

RF	Descrição	Evidência no projeto	Validação/ajuste necessário
RF01	Permitir a criação de pedido de medicamento pelo cliente/canal.	createOrder(), addItem() e finishOrder() em PharmacyApplicationService; Order.java.	Definir estados válidos do pedido e bloquear finalização em erros críticos.
RF02	Controlar estoque e dar baixa no lote ao adicionar item ao pedido.	InMemoryStockRepository.findByMedicine() ; StockLot.quantity; addItem() decrementa o primeiro lote.	Validar quantidade maior que zero e data de validade do lote.
RF03	Tratar medicamentos que exigem receita.	Medicine.prescriptionRequired; addItem() recebe prescriptionNumber.	Hoje apenas imprime aviso e continua. Deve validar antes da compra ou enviar para revisão com estado explícito.
RF04	Processar pagamento, entrega e notificação do pedido.	PaymentLegacyGateway, DeliveryPartnerApi, WhatsappLegacyApi e finishOrder().	Entrega e notificação devem depender do resultado correto do fluxo e ter tratamento de falha.

4. Requisitos não funcionais

RNF	Qualidade	Formulação verificável	Métrica / limite / cenário	Forma de validação
RNF01	Manutenibilidade	Separar responsabilidades do fluxo de pedido, estoque, receita, pagamento, entrega e notificação em componentes com fronteiras claras.	Nenhuma classe de aplicação deve concentrar todas as integrações e regras principais.	Revisão de dependências e inspeção das classes de serviço.
RNF02	Segurança	Pagamento e receita não devem ser processados sem validação mínima e não devem expor dados sensíveis em logs.	Pedidos com medicamento de receita sem número válido não podem seguir como compra finalizada.	Teste de cenário com prescriptionNumber vazio e análise dos logs.
RNF03	Disponibilidade / confiabilidade	Falha em notificação ou auditoria não deve impedir o registro do pedido pago, mas deve ser rastreável para reproprocessamento.	Fluxo principal não deve depender diretamente de WhatsApp/Audit síncronos.	Simular falha no notificador e verificar manutenção do estado do pedido.
RNF04	Desempenho	O fluxo de compra deve evitar chamadas externas desnecessárias antes das validações de negócio.	Em cenário normal, 95% das operações locais devem finalizar em até 2 segundos; integrações devem ser isoladas.	Teste manual/automatizado de tempo e revisão da ordem do fluxo.

5. Relação RNF x parte afetada

RNF	Parte afetada	Por que é afetada?	Evidência
RNF01	Camada de aplicação / serviços	PharmacyApplicationService concentra muitas decisões. Isso dificulta manutenção, teste e evolução.	service/PharmacyApplicationService.java possui estoque, receita, pagamento, entrega, WhatsApp e eventos.
RNF02	Domínio, pagamento e receita	O fluxo permite continuar sem receita válida e executa pagamento/entrega sem fronteira clara de validação.	addItem() apenas imprime WARNING; finishOrder() sempre chama delivery.dispatch().

RNF03	Notificação, auditoria e eventos	OrderPublisher mantém apenas um observer, substituindo o anterior. Notificação/auditoria não estão robustas para falhas.	subscribe(new CustomerNotificationObserver()) seguido de subscribe(new AuditObserver()).
RNF04	Integrações e persistência	Chamadas externas simuladas estão acopladas ao serviço central; qualquer lentidão/falha impacta o fluxo.	PaymentLegacyGateway, DeliveryPartnerApi e WhatsappLegacyApi são instanciadas dentro do serviço.

6. Problema arquitetural encontrado

Problema identificado: o serviço de aplicação central possui acoplamento alto e responsabilidades concentradas. Ele cria dependências diretas para APIs legadas, controla estoque, trata receita, calcula desconto, processa pagamento, aciona entrega, envia WhatsApp e publica eventos. Além disso, o mecanismo de observer permite somente um observador ativo, fazendo com que a auditoria substitua a notificação do cliente.

Evidência: em PharmacyApplicationService há campos diretos para PaymentLegacyGateway, DeliveryPartnerApi, WhatsappLegacyApi e DiscountCalculator. No construtor, publisher.subscribe(new CustomerNotificationObserver()) é seguido por publisher.subscribe(new AuditObserver()), substituindo o observador anterior. Em finishOrder(), a entrega é acionada mesmo quando o pagamento pode falhar.

Consequência possível: aumento de dificuldade para testar e modificar o sistema; risco de pedido seguir com receita ausente; risco de despacho após pagamento recusado; perda de notificação ou auditoria; e baixa clareza para evoluir integrações reais.

7. Matriz de decisão arquitetural

Alternativas analisadas: A) manter monólito em camadas simples; B) evoluir para monólito modular em camadas com portas/adapters e eventos internos; C) migrar para microserviços/SOA separados por domínio.

Critério	Peso	Nota A	P×N A	Nota B	P×N B	Nota C	P×N C
Desempenho	3	3	9	4	12	4	12
Segurança	3	2	6	4	12	5	15
Manutenibilidade	3	2	6	5	15	4	12
Disponibilidade / confiabilidade	3	2	6	4	12	5	15
Complexidade operacional	2	5	10	4	8	1	2
Adequação ao estágio do projeto	2	4	8	5	10	2	4
TOTAL			45		69		60

Resultado: Alternativa A = 45 pontos; Alternativa B = 69 pontos; Alternativa C = 60 pontos. A maior pontuação foi da Alternativa B, pois melhora os pontos críticos sem exigir a complexidade operacional de microserviços.

7.1 Justificativa das notas

Critério	Alternativa	Nota	Justificativa	Evidência no projeto
Desempenho	A	3	Atende de forma aceitável no cenário didático em memória, mas integrações diretas podem atrasar o fluxo.	Fluxo em PharmacyApplicationService chama pagamento, entrega e WhatsApp no mesmo método.
Desempenho	B	4	Melhora ao isolar integrações por portas e permitir eventos para notificação/auditoria sem bloquear o fluxo principal.	Adapters já existem e podem virar fronteiras formais.
Desempenho	C	4	Pode escalar partes separadas, mas adiciona custo de rede e comunicação entre serviços.	Projeto atual não possui contratos nem bancos separados.
Segurança	A	2	Mantém riscos: receita ausente apenas gera aviso e despacho pode ocorrer após erro de pagamento.	addItem() imprime WARNING; finishOrder() chama delivery.dispatch().
Segurança	B	4	Permite centralizar validações no domínio e isolar pagamento em interface, reduzindo exposição e acoplamento.	Medicine.prescriptionRequired e PaymentAdapter podem ser usados em fronteiras claras.
Segurança	C	5	Permite controles independentes por serviço, mas exige autenticação entre serviços e governança maior.	Pagamentos, pedidos e receitas poderiam ser serviços separados.
Manutenibilidade	A	2	Responsabilidades continuam concentradas em uma classe central.	PharmacyApplicationService reúne muitas decisões.
Manutenibilidade	B	5	Divide responsabilidades mantendo um único projeto, facilitando refatoração incremental.	Pacotes model, repository, service, legacy e patterns já dão base para modularização.
Manutenibilidade	C	4	Separação forte, mas aumenta manutenção de vários serviços, contratos e deploys.	Não há infraestrutura de microsserviços no repositório.
Disponibilidade	A	2	Falhas em integrações e observer afetam rastreabilidade e podem perder notificação.	OrderPublisher guarda um único observer.
Disponibilidade	B	4	Eventos internos permitem desacoplar notificação e auditoria do fluxo principal com tratamento de falha.	Observer pode evoluir para lista de observadores/event handlers.
Disponibilidade	C	5	Serviços isolados suportam falhas parciais, porém exigem observabilidade e tratamento distribuído.	Arquitetura atual é local e sem mensageria real.
Complexidade operacional	A	5	É a alternativa mais simples de executar e depurar.	Execução por javac e Main.
Complexidade operacional	B	4	Aumenta organização interna, mas continua um único sistema Java.	Não exige múltiplos deploys.

Complexidade operacional	C	1	Exige infraestrutura, rede, monitoramento, contratos e persistência por serviço.	Projeto atual não possui suporte operacional para isso.
Adequação ao estágio	A	4	Adequada para estudo inicial, mas insuficiente para os RNFs levantados.	Legado executa, porém documentação diz que há decisões incompletas.
Adequação ao estágio	B	5	É coerente com o projeto em evolução e permite melhorar sem criar tecnologia desnecessária.	Aula orienta justificar evolução sem aumentar complexidade sem necessidade.
Adequação ao estágio	C	2	É interessante conceitualmente, mas precoce para o tamanho e maturidade atual.	Não há API REST, banco real, contratos ou automação de deploy.

8. Decisão, trade-off e proposta

Decisão: propor evolução arquitetural para a Alternativa B - monólito modular em camadas, com portas/adapters para integrações externas e eventos internos para notificação e auditoria.

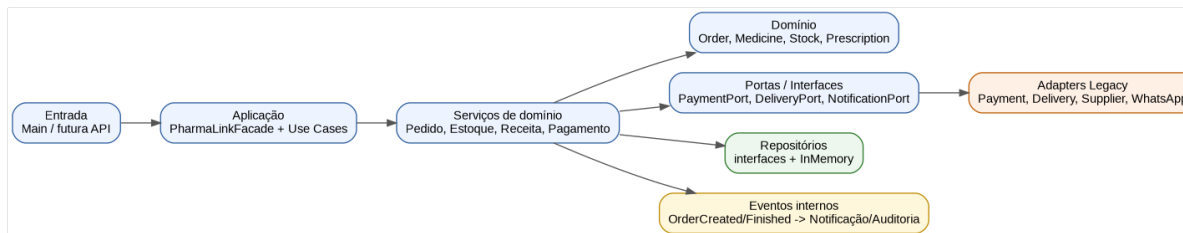
Justificativa: a alternativa B responde melhor ao problema real do PharmaLink. Ela preserva a simplicidade de execução do projeto, mas cria fronteiras mais claras entre aplicação, domínio, repositórios e integrações. Também permite tratar RNFs importantes, como segurança, manutenibilidade, disponibilidade e desempenho, sem assumir a complexidade de microserviços neste momento.

Ganho: maior separação de responsabilidades, menor acoplamento com APIs legadas, melhor testabilidade e base mais segura para evoluir pagamento, receita, entrega e notificação.

Custo/consequência: será necessário refatorar classes, criar interfaces, ajustar o fluxo de pedido e escrever testes para validar os cenários de receita, pagamento, estoque e notificação.

Trade-off aceito pelo grupo: aceitar um pouco mais de estrutura interna agora para reduzir riscos de manutenção e segurança, mantendo o sistema em uma única aplicação para não elevar a complexidade operacional.

8.1 Diagrama da arquitetura proposta



Melhorias internas recomendadas

- Criar serviços menores: OrderService, StockService, PrescriptionService, PaymentService, DeliveryService e NotificationService.
- Criar portas/interfaces para PaymentPort, DeliveryPort, SupplierPort e NotificationPort, deixando as classes legacy apenas como adapters.
- Trocar o OrderPublisher para permitir múltiplos observadores/event handlers, preservando notificação e auditoria ao mesmo tempo.
- Bloquear ou colocar em revisão pedidos com medicamento de receita sem prescrição válida.
- Impedir despacho quando o pagamento falhar e registrar estados explícitos como CREATED, WAITING_PRESCRIPTION, PAID, PAYMENT_ERROR e DISPATCHED.

9. Conclusão do grupo

O grupo identificou que o PharmaLink não está pronto para microsserviços, apesar de possuir uma estrutura inicial em camadas e padrões de projeto. A comparação mostrou que manter a arquitetura atual preserva simplicidade, mas não resolve os principais riscos de segurança, manutenibilidade e confiabilidade. Microsserviços oferecem isolamento, porém adicionam complexidade incompatível com o estágio atual. Por isso, a decisão adequada é evoluir para um monólito modular em camadas, com portas/adapters e eventos internos. Essa decisão atende melhor aos RNFs analisados e permite evolução gradual do projeto com evidências concretas do próprio código.

Checklist antes da entrega

- ☒ Sumário incluído e atualizado.
- ☒ Arquitetura atual descrita com evidências.
- ☒ Diagrama da arquitetura atual incluído.
- ☒ Pelo menos 2 RF analisados com evidências.
- ☒ Pelo menos 3 RNF refinados e verificáveis.
- ☒ Tabela RNF x parte afetada preenchida.
- ☒ Problema arquitetural real identificado.
- ☒ Três alternativas comparadas na matriz.
- ☒ Pesos, notas, cálculos e justificativas preenchidos.
- ☒ Trade-off e consequência da decisão explicados.
- ☒ Diagrama atual x proposta coerente.
- ☒ Arquivo final exportado em PDF para entrega no Classroom.